Đồ án 2

ĐỀ CƯƠNG NGHIÊN CỨU

TÊN ĐỀ TÀI
Mini-NPU: Bộ tăng tốc RISC-V lấy cảm hứng từ CoralNPU với MAC engine int8 tách rời và clock gating mức module, được kiểm chứng bằng môi trường UVM tự xây dựng

Mục	Thông tin
Sinh viên / Nhóm	Nguyễn Hoàng Công Tấn / Nguyễn Thế Bảo
MSSV	23161064 / 23144002
Chương trình / Khóa	Công nghệ kỹ thuật Điện tử - Viễn thông, lớp 23161FIE
Giảng viên hướng dẫn	TS. Phạm Văn Khoa
Lĩnh vực nghiên cứu	Bộ tăng tốc AI cho thiết bị biên (NPU RISC-V), thiết kế RTL và kiểm chứng UVM, thiết kế số công suất thấp
Thời gian thực hiện	16 tuần (12/10/2026 – 31/01/2027)
Ngày nộp	11/11/2026

Logic nghiên cứu bắt buộc

Giai đoạn	Câu hỏi cốt lõi	Bằng chứng yêu cầu
1. Vấn đề kỹ thuật	Chính xác thì điều gì sai, bị hạn chế, tốn kém, chậm, thiếu chính xác, không an toàn hoặc chưa được giải quyết về mặt kỹ thuật?	Vấn đề được định lượng, ràng buộc, thước đo, câu hỏi nghiên cứu
2. Công trình liên quan	Các nghiên cứu hoặc hệ thống hiện có đã giải quyết vấn đề này như thế nào?	So sánh có phản biện, kết quả, hạn chế, khoảng trống nghiên cứu
3. Ý tưởng đề xuất	Ý tưởng mới hoặc được điều chỉnh nào được đề xuất để lấp khoảng trống?	Khái niệm hệ thống, kiến trúc, tính mới, giả thuyết
4. Phương pháp luận	Ý tưởng sẽ được hiện thực và đánh giá khách quan như thế nào?	Kế hoạch hiện thực, baseline, thí nghiệm, thước đo, phân tích
5. Kết quả kỳ vọng	Những kết quả đo được và sản phẩm bàn giao nào được kỳ vọng?	Hiệu năng mục tiêu, đóng góp, prototype/báo cáo/demo

Chuẩn viết khuyến nghị

- Dùng thuật ngữ kỹ thuật chính xác và các chỉ số hiệu năng đo được thay cho những nhận định chung chung như “tốt hơn”, “nhanh hơn” hay “hiệu quả hơn”.
- Dùng các nguồn được bình duyệt và đáng tin cậy về mặt kỹ thuật. Ưu tiên công trình gần đây cho hiện trạng kỹ thuật, đồng thời giữ lại các tài liệu nền tảng khi cần.
- Với mỗi nhận định quan trọng, đưa ra bằng chứng qua trích dẫn, phép đo, dữ liệu benchmark hoặc một giả định được nêu rõ.
- Định nghĩa mọi từ viết tắt ở lần dùng đầu tiên và dùng nhất quán ký hiệu, đơn vị, tên dataset, đặc tả phần cứng và điều kiện đánh giá.
- Giải pháp đề xuất phải trực tiếp giải quyết khoảng trống nghiên cứu rút ra từ các công trình liên quan; các thước đo đánh giá phải phản ánh trực tiếp các vấn đề kỹ thuật nêu ở Phần 1.

Độ dài đề xuất

Thành phần	Độ dài đề xuất
Các phần chính 1-5	6-10 trang
Hình / Bảng	Tùy theo nhu cầu làm rõ kỹ thuật
Tài liệu tham khảo	Thông thường từ 10 nguồn đáng tin cậy trở lên cho một đề tài hướng nghiên cứu
Phụ lục	Tùy chọn; chi tiết hiện thực, bảng mở rộng, bảng rủi ro, v.v.

1 VẤN ĐỀ KỸ THUẬT / MÔ TẢ BÀI TOÁN

Xác định bài toán kỹ thuật bằng các thuật ngữ đo được và có cơ sở kỹ thuật.

1.1 Bối cảnh và ngữ cảnh kỹ thuật

Suy luận machine learning đang chuyển từ trung tâm dữ liệu sang các thiết bị chạy pin và bị giới hạn chi phí như thiết bị đeo, cảm biến và thiết bị nhà thông minh. Trên các thiết bị này, khối lượng tính toán bị chi phối bởi phép nhân-tích lũy (multiply-accumulate, MAC) trên số nguyên nhỏ, và lượng tử hóa sau huấn luyện xuống số nguyên 8 bit (int8) giữ được độ chính xác trong khi cho phép suy luận chỉ dùng số học số nguyên [1]. Các bộ tăng tốc chuyên dụng khai thác tính đều đặn này: Tensor Processing Unit (TPU) dùng một matrix unit 8 bit lớn [2],

Eyeriss giảm năng lượng di chuyển dữ liệu bằng một dataflow riêng [3], và khảo sát trong [4] cho thấy năng lượng bị chi phối bởi mức tái sử dụng dữ liệu và mức độ các đơn vị tính toán được giữ bận. Nhận định rằng phép tính tốn năng lượng ít hơn nhiều so với việc di chuyển dữ liệu [5] khiến các bộ nhớ gắn chặt (tightly-coupled memory) nhỏ và cổng toán hạng rộng trở nên hấp dẫn cho một neural processing unit (NPU) ở thiết bị biên.

Các dự án mã nguồn mở giúp việc nghiên cứu những kiến trúc này trở nên khả thi. Coral NPU của Google kết hợp một scalar core RISC-V nhỏ với các vector unit, matrix unit và bộ nhớ gắn chặt [6]; Gemmini sinh ra các bộ tăng tốc systolic-array gắn vào các core RISC-V [7]. Dự án này theo cùng ý tưởng ở quy mô mà một nhóm hai người có thể thiết kế, kiểm chứng và giải thích trọn vẹn. Một core RV32IM 5 tầng (với các extension Zicsr, Zifencei và Zbb [8]) được mở rộng thêm bốn custom instruction là LOAD_WEIGHT, LOAD_INPUT, COMPUTE và READ_RESULT, đẩy các command qua một FIFO 8 phần tử tới một MAC engine outer-product int8 chạy tách rời với N×M×K = 4×4×4 (16 processing element, mỗi PE là một dot product 4 phần tử, 64 MAC mỗi cycle). Core và engine dùng chung một bộ nhớ dữ liệu hai cổng 64 KB (DTCM). Workload mục tiêu là một multilayer perceptron (MLP) int8 784-64-32-10 trên MNIST [9]. Đề tài thuộc Computer Engineering vì nó gộp mở rộng tập lệnh, vi kiến trúc, thiết kế hệ thống bộ nhớ, firmware bare-metal và kiểm chứng phần cứng trong một hệ thống, đúng chủ đề đồng thiết kế phần cứng/phần mềm của tính toán chuyên dụng [10].

Trong một hệ thống như vậy, engine ở trạng thái idle mỗi khi core chạy mã không dùng đến nó (tạo địa chỉ, điều khiển vòng lặp, hàm kích hoạt, vào/ra). Trong thiết kế RTL thông thường, clock của engine vẫn đến mọi flip-flop của các operand buffer (2 × 256 × 128 bit), PE array và accumulator ở mỗi cycle, nên cây clock và các thanh ghi tiếp tục chuyển trạng thái. Công suất động tăng theo switching activity, điện dung, bình phương điện áp nguồn và tần số [11], và một clock chạy tự do chuyển trạng thái ở mọi cycle; clock gating là biện pháp chuẩn cho các thanh ghi idle [12][13][14]. Gating ở mức module với tín hiệu enable suy ra từ trạng thái phần cứng đơn giản về nguyên lý, nhưng nó tạo ra hai nghĩa vụ chính là đối tượng của dự án này. Thứ nhất, gate không được làm thay đổi hành vi hay timing, vì bất kỳ cycle nào bị thêm vào hoặc mất đi đều làm hỏng phần mềm có đếm cycle. Thứ hai, lợi ích của nó phải được định lượng một cách trung thực: switching activity lấy từ mô phỏng RTL là một ước lượng của công suất, không phải phép đo công suất [15].

Thực hành công nghiệp kiểm chứng các khối như vậy bằng môi trường Universal Verification Methodology (UVM) với stimulus constrained-random, golden reference model, assertion và functional coverage [16][17]. Điều này hiếm khi được làm ở quy mô sinh viên, nên dự án tự xây môi trường đó (ba môi trường UVM cho engine, core và toàn SoC) và dùng nó làm bằng chứng rằng thiết kế có gating là đúng.

1.2 Phát biểu bài toán kỹ thuật

Bài toán kỹ thuật mà dự án giải quyết là: một MAC engine int8 nhỏ gắn với RISC-V làm lãng phí switching activity khi idle, và biện pháp khắc phục là clock gating chỉ đáng tin nếu thiết kế có gating được chứng minh hoạt động đúng như thiết kế không gating. Thiết kế không gating (D0) cấp clock cho toàn bộ datapath của engine ở mọi cycle bất kể tỉ lệ idle; thiết kế có gating (D1) phải loại bỏ activity đó mà không thêm một cycle nào, không đổi kết quả nào và không làm hỏng command đang thực thi. Vì vậy dự án phải (i) thiết lập một baseline chức năng đã được kiểm chứng, khớp bit-exact với các golden model độc lập (D0), (ii) chứng minh D1 giống D0 cả về cycle lẫn kết quả, và (iii) đo mức giảm switching activity theo hàm của idle ratio của engine trên các workload tổng hợp và MNIST.

1.3 Ràng buộc kỹ thuật và điều kiện vận hành

Ràng buộc / Điều kiện	Giá trị hiện tại hoặc yêu cầu	Ý nghĩa kỹ thuật
Tài nguyên tính toán	QuestaSim 10.7c (UVM 1.2, DPI-C, coverage); Verilator 5.020 và RISC-V GNU toolchain trong WSL2 (Ubuntu 24.04); Icarus Verilog 12 và GitHub Actions CI; Python 3 với numpy và PyTorch	QuestaSim chạy flow UVM và tạo các file dump activity. Bộ tính năng của nó (randomize, covergroup, DPI-C) đã được kiểm ở tuần 1. Verilator chỉ dùng để lint.

Ràng buộc / Điều kiện	Giá trị hiện tại hoặc yêu cầu	Ý nghĩa kỹ thuật
Bộ nhớ / Lưu trữ	ITCM 16 KB; DTCM 64 KB (4 bank × 32 bit); WBUF và IBUF mỗi cái 256 × 128 bit; command FIFO 8 phần tử; KMAX = 1024 phần tử; vùng dữ liệu của model 56,816 B	Toàn bộ MLP nằm vừa trong DTCM, nên không cần DMA hay bộ nhớ ngoài.
Độ trễ / Yêu cầu thời gian thực	Không có deadline thời gian thực. Mỗi command của engine tốn K/4 + 2 cycle bận (chặn trên K/4 + 3). D1 phải thêm 0 cycle vào mọi chương trình.	Hành vi giống hệt nhau từng cycle giữa D0 và D1 (SC3) là tiêu chí khiến gating chấp nhận được với phần mềm đọc mcycle.
Năng lượng / Công suất	Ước lượng switching activity từ mô phỏng RTL: số toggle trên clk_mac (M1) và trên mọi net của engine (M2), tính trên 1,000 cycle	Thước đo chính của nghiên cứu gating. Không dùng synthesis hay thư viện công nghệ, nên kết quả không bao giờ được gọi là “công suất đo được”.
Độ chính xác / Chất lượng	MLP int8: 97.43 % trên 10,000 ảnh test MNIST (model float 97.48 %); logit của RTL khớp bit-exact với golden model trên 1,000 ảnh đầu	Tiêu chí thành công là bit-exact chứ không phải accuracy; accuracy cho thấy lượng tử hóa không phải yếu tố giới hạn.
Truyền thông / Băng thông	DTCM port A 32 bit đọc/ghi (core); port B 128 bit chỉ đọc (engine), mỗi cycle một quad gồm bốn word	Port B cấp một entry WBUF/IBUF mỗi cycle, giữ cho PE array 64 MAC luôn có dữ liệu.
Chi phí / Diện tích / Mức dùng tài nguyên	N×M×K = 4×4×4: 16 PE dot4 (64 bộ nhân int8); 8×8 chỉ làm nếu còn thời gian sau khi đạt các tiêu chí cốt lõi	Xác định kích thước engine. Diện tích không được đánh giá vì synthesis nằm ngoài dự án.
Bảo mật / Riêng tư / An toàn	Không phải mục tiêu của dự án. Engine bỏ các command sai và ghi lại các error bit dạng sticky (err_k, err_addr, err_illegal, err_align).	Việc xử lý lỗi được đặc tả và kiểm chứng, nhưng không thực hiện phân tích bảo mật.

1.4 Câu hỏi nghiên cứu

RQ1. Clock gating mức module điều khiển bởi idle do phần cứng phát hiện có giữ Mini-NPU giống hệt thiết kế không gating cả về cycle lẫn kết quả (D1 so với D0) trên các workload directed, random, tổng hợp và MNIST hay không?

RQ2. Gating giảm switching activity của MAC engine (số toggle trên clk_mac và trên mọi net của engine) được bao nhiêu theo hàm của idle ratio của engine, và workload MNIST có đi theo xu hướng dự đoán từ các workload tổng hợp hay không?

RQ3. Một môi trường UVM tự xây dựng (stimulus constrained-random, golden model C không có thời gian gọi qua DPI-C, assertion và functional coverage) có đóng được việc kiểm chứng engine và toàn SoC theo các mục tiêu của dự án trong thời gian thực hiện hay không?

Tiêu chí thành công

Tiêu chí	Thước đo	Điều kiện chấp nhận / Mục tiêu
SC1 Tính đúng chức năng	Số mismatch ở scoreboard; số assertion fail	Cả hai bằng 0 với mọi directed test và random test đã định nghĩa, trên D0 và D1
SC2 Coverage	Các bin functional coverage khai báo trong coverage plan	100 % bin đã khai báo trên D0 trước khi bắt đầu làm gating (tuần 10); mọi ngoại lệ phải có lý do bằng văn bản
SC3 Tương đương D0/D1	SHA-256 của retire log; mcycle và minstret cuối; output dump	Giống hệt với cùng chương trình và seed. Dự phòng: giống về kết quả, kèm lý do được ghi lại.
SC4 Switching activity	% thay đổi của M1 và M2, D1 so với D0	Báo cáo ở idle ratio 10–90 % (tổng hợp) và cho MNIST; không đặt trước phần trăm mục tiêu
SC5 MNIST	Logit và nhãn của RTL so với golden model	Bit-exact trên 1,000 ảnh test đầu (D0 và D1); lần chạy đủ 10,000 ảnh trên D0 tái tạo đúng accuracy của golden

Tiêu chí	Thước đo	Điều kiện chấp nhận / Mục tiêu
SC6 Tái lập	Câu lệnh và phiên bản được ghi lại	Một lệnh cho mỗi flow tái tạo dữ liệu regression, coverage và activity; phiên bản công cụ và seed được ghi lại

2 CÁC GIẢI PHÁP HIỆN CÓ VÀ CÔNG TRÌNH LIÊN QUAN

Xác lập hiện trạng kỹ thuật và rút ra một khoảng trống nghiên cứu có cơ sở.

2.1 Phạm vi tài liệu và chiến lược tìm kiếm

Phần tổng quan bao gồm bốn chủ đề tương ứng với bốn đóng góp của dự án: (i) kiến trúc và bộ sinh bộ tăng tốc ghép một matrix engine với một host core; (ii) chi phí năng lượng của phép tính, di chuyển dữ liệu và phần cứng idle; (iii) clock gating và ước lượng công suất dựa trên activity; và (iv) lượng tử hóa số nguyên cùng phương pháp kiểm chứng cần thiết để chứng minh một bộ tăng tốc là đúng. Các công trình được bình duyệt giai đoạn 2014–2021 được dùng cho bộ tăng tốc và lượng tử hóa, các bài nền tảng giai đoạn 1994–2000 được giữ lại cho clock gating và ước lượng công suất, còn repository Coral NPU mã nguồn mở (commit 382b5c1) cùng các developer guide của nó, đọc ngày 15/10/2026, đại diện cho NPU RISC-V mở hiện nay. Các tiêu chuẩn (IEEE 1800, 1800.2 và RISC-V ISA manual) cùng hai giáo trình [11][18] xác định thuật ngữ.

2.2 Tóm tắt các công trình liên quan

Tài liệu	Vấn đề / Bối cảnh	Phương pháp / Kiến trúc	Kết quả chính	Hạn chế liên quan đến dự án này
[2] Jouppi et al. (2017)	Chi phí suy luận mạng neural ở trung tâm dữ liệu	Matrix unit systolic 256×256 gồm các MAC 8 bit với bộ nhớ on-chip do phần mềm quản lý	Báo cáo hiệu năng cao hơn khoảng 15–30× và TOPS/W cao hơn 30–80× so với CPU và GPU cùng thời trên workload sản xuất	Quy mô trung tâm dữ liệu và chức năng cố định; không có host RISC-V, không có nghiên cứu idle-power hay clock gating ở mức module
[3] Chen et al. (2017)	Năng lượng bị chi phối bởi di chuyển dữ liệu trong bộ tăng tốc CNN	Dataflow row-stationary trên 168 PE, công nghệ CMOS 65 nm	Hiệu quả năng lượng đo trên silicon nhờ tái sử dụng dữ liệu ở các lớp convolution của AlexNet	Chip CNN chức năng cố định; công suất được quản lý bằng dataflow chứ không bằng gating theo idle do phần cứng phát hiện; không có RTL mở hay flow kiểm chứng
[7] Genc et al. (2021)	Đánh giá bộ tăng tốc DNN cùng với hệ thống host của chúng	Bộ sinh Chisel cho bộ tăng tốc systolic-array gắn vào core RISC-V, kèm software stack	Khảo sát không gian thiết kế toàn hệ thống giữa bộ tăng tốc và host	Hướng tới khảo sát không gian thiết kế; gating và tính tương đương cycle của biến thể có gating không phải trọng tâm; code base lớn
[6] Google, Coral NPU	NPU RISC-V mở cho edge AI	Scalar core in-order, vector unit và matrix unit outer-product; thường dùng MAC array 8×8×4; có test UVM và cocotb	Các mục tiêu thiết kế trong guide (ví dụ mức công suất 10 mW); clock gate toàn core theo WFI hoặc CSR	Là mục tiêu chứ không phải phép đo; matrix unit được đánh dấu thử nghiệm trong repository; gating ở mức toàn core, không phải mức engine
[12] Wu et al. (2000)	Các thanh ghi giữ nguyên giá trị vẫn làm tốn công suất clock	Suy ra điều kiện clock gating từ trạng thái và activity của chính mạch	Giảm công suất trên các mạch tuần tự chuẩn	Mức gate, mạch tuần tự nhỏ; không có bộ tăng tốc, không kiểm tra tương đương ở mức SoC
[13] Benini và De Micheli (1996)	Các self-loop idle trong máy trạng thái hữu hạn	Tổng hợp FSM có gated-clock từ một activation function	Tiết kiệm công suất nhờ dừng clock ở các trạng thái idle	Áp dụng cho bộ điều khiển FSM, không phải datapath có operand buffer và accumulator
[1] Jacob et al. (2018)	Chạy mạng neural chỉ bằng số học số nguyên	Weight và activation int8, tích lũy int32, requantization dấu phẩy cố định	Cải thiện đánh đổi accuracy–độ trễ cho suy luận MobileNet trên CPU di động	Là phần mềm trên CPU; requantization bằng phần cứng và golden model bit-exact cho phần cứng để lại cho người hiện thực

2.3 So sánh phản biện các hướng tiếp cận hiện có

Hướng tiếp cận	Ý tưởng chính	Ưu điểm	Hạn chế	Điều kiện đánh giá / Hiệu năng
Hướng A Bộ tăng tốc dataflow và systolic lớn ([2][3][7])	Tối đa hóa mức sử dụng và tái sử dụng dữ liệu bằng một array và dataflow chuyên dụng	Thông lượng trên mỗi watt cao khi array bận	Thời gian idle giữa các đợt tính không phải mục tiêu của chúng; kích thước và độ phức tạp vượt quá một dự án hai người	ASIC hoặc FPGA, node và workload khác nhau; các con số không so sánh trực tiếp được với nhau hay với dự án này
Hướng B NPU RISC-V mở ([6])	Scalar core cùng matrix engine tách rời; gating toàn core theo WFI hoặc CSR	Hệ thống mở hoàn chỉnh có software stack; có cấu trúc UVM để tham khảo	Độ mịn gating thô; số liệu công bố là mục tiêu thiết kế; code base Chisel rất lớn	Mục tiêu công suất ở mức guide và tài liệu thiết kế; không so sánh trực tiếp được với số toggle của RTL
Hướng C Clock gating mức gate và tổng hợp FSM gated-clock ([12][13])	Tắt clock của các thanh ghi có trạng thái kế tiếp bằng trạng thái hiện tại	Tự động, được hiểu rõ, chi phí diện tích nhỏ	Nghiên cứu trên các mạch tuần tự nhỏ; không có phương pháp kiểm tra tương đương cycle cho cả SoC	Mạch benchmark ở mức gate; mức tiết kiệm tính theo phần trăm của baseline riêng. Không so sánh trực tiếp được.
Hướng D Suy luận lượng tử hóa chỉ dùng số nguyên ([1])	Datapath int8 với accumulator int32 và rescale dấu phẩy cố định	Giữ được accuracy; ánh xạ tốt lên các MAC array nhỏ	Tham chiếu là phần mềm; requantization bằng phần cứng cần golden model riêng	MobileNet trên ImageNet chạy trên CPU di động; không so sánh trực tiếp được với MLP MNIST

2.4 Khoảng trống nghiên cứu được xác định

Các bộ tăng tốc hiện có đạt hiệu quả năng lượng chủ yếu nhờ dataflow và mức sử dụng [2][3][4]. Thời gian idle của một bộ tăng tốc tải nhẹ nằm sau một scalar core được xử lý ở mức thô, bằng cách gating toàn core [6], hoặc chỉ ở mức gate trên các mạch nhỏ [12][13]. Trong các nguồn đã xem xét ở đây, không có công trình nào kết hợp cùng lúc ba yếu tố mà dự án này cần: (a) gating mức module, theo idle do phần cứng phát hiện, cho một engine int8 nhỏ gắn với RISC-V; (b) một tiêu chí tương đương D0/D1 giống hệt nhau từng cycle được kiểm tra bằng hash của retire trace thay vì được giả định; và (c) một quan hệ đo được giữa mức giảm switching activity và idle ratio của engine trên cả workload tổng hợp lẫn workload thật (MNIST) đối chiếu với một golden model bit-exact.

Đây là khoảng trống về bằng chứng, không phải tuyên bố về một mạch gating mới. Clock gate dạng latch cộng AND là kiến thức giáo trình [14]. Số toggle cũng không phải công suất: nó bỏ qua điện dung, glitch và chi phí riêng của gate [15], nên dự án báo cáo chúng như các ước lượng và tách khỏi bằng chứng chức năng.

2.5 Tuyên bố định vị

So với các hướng tiếp cận đã xem xét, dự án này không đề xuất một kỹ thuật mức mạch mới [12][13] hay một bộ tăng tốc lớn hơn [2][7]. Đóng góp của nó là một nghiên cứu quy mô nhỏ nhưng hoàn chỉnh và tái lập được: một NPU được thiết kế và kiểm chứng từ tập lệnh xuống RTL, được gating ở mức module, và được chứng minh bằng bằng chứng là tương đương với bản song sinh không gating. Nó khác [6] ở chỗ gating MAC engine theo idle do phần cứng phát hiện thay vì gating toàn core theo một tín hiệu phần mềm, và ở chỗ tự xây core 5 tầng, golden model và các môi trường UVM để mọi quyết định thiết kế đều giải thích được. Việc đánh giá dùng mô phỏng RTL trên QuestaSim, quét idle ratio (10, 25, 50, 75 và 90 %) và đo số toggle trên 1,000 cycle, lấy thiết kế không gating làm chuẩn đối chiếu.

3 Ý TƯỞNG ĐỀ XUẤT / GIẢI PHÁP ĐỀ XUẤT

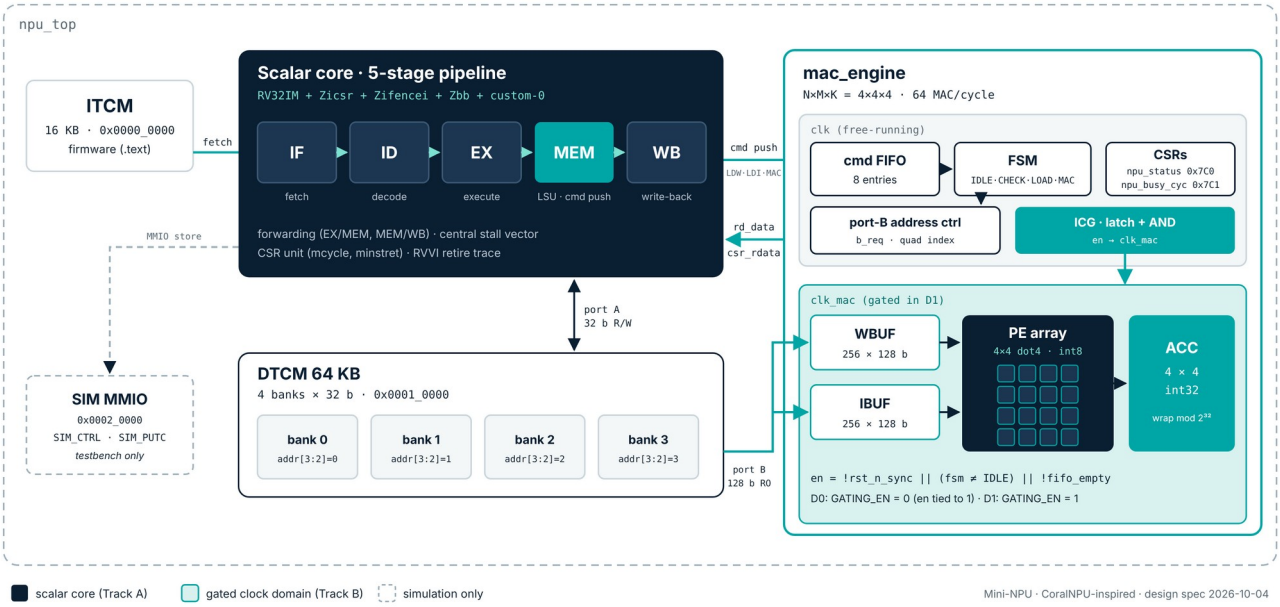
Mô tả khái niệm giải pháp, kiến trúc, tính mới và các giả thuyết kiểm chứng được.

3.1 Ý tưởng kỹ thuật cốt lõi

Dự án này giới thiệu một NPU RISC-V lấy cảm hứng từ CoralNPU, trong đó bốn custom-0 instruction đẩy command qua một FIFO 8 phần tử tới một MAC engine outer-product int8 chạy tách rời (4×4×4 = 64 MAC mỗi cycle), và trong đó clock của datapath của engine (operand buffer, PE array, accumulator) được gating bằng một cell latch cộng AND. Tín hiệu enable được bật khi FSM của engine không idle, FIFO của nó không rỗng, hoặc đang reset. Vì enable chỉ phụ thuộc vào trạng thái được lưu trong thanh ghi, một command được đẩy vào ở cycle t sẽ mở gate kịp cho cạnh clock đầu tiên mà datapath cần, tức cycle t+2, nên gating không thêm cycle nào và D1 có thể giống hệt D0 từng cycle. Thiết kế được kiểm chứng bằng các môi trường UVM tự xây và một MLP MNIST khớp bit-exact, và lợi ích của gating được định lượng bằng số toggle theo idle ratio của engine. Điều này giải quyết khoảng trống ở Mục 2.4: gating ở mức engine, tương đương được chứng minh bằng bằng chứng, và activity được đo trên workload thật và tổng hợp.

3.2 Kiến trúc hệ thống đề xuất

Thành phần kiến trúc	Mô tả / Giao diện / Chức năng
Đầu vào / Thu nhận dữ liệu	Firmware bare-metal (C và assembly, custom instruction qua các wrapper `.insn`) và dữ liệu MLP int8 được nạp vào ITCM và DTCM bằng backdoor `$readmemh` trước reset. Pixel MNIST p được lượng tử hóa thành (p·254 + 255) // 510 trong khoảng 0..127; model 784-64-32-10 được huấn luyện trong notebook và xuất ra file `.mem` kèm hash SHA-256.
Tiền xử lý / Lớp giao diện	Core 5 tầng RV32IM + Zicsr + Zifencei + Zbb giải mã các custom-0 instruction và đẩy các command LOAD_WEIGHT, LOAD_INPUT, COMPUTE từ tầng MEM vào command FIFO 8 phần tử; READ_RESULT stall cho đến khi engine xả hết.
Module cốt lõi đề xuất	`mac_engine`: FSM (IDLE, CHECK, LOAD, MAC); operand buffer WBUF và IBUF (256 × 128 bit); 16 PE dot4 và một tile accumulator int32 4×4 (wrap theo modulo 2^{32}); các CSR npu_status và npu_busy_cyc. Toán hạng đến từ DTCM port B (128 bit, chỉ đọc).
Tối ưu / Điều khiển / Lập lịch	`npu_clk_gate` (latch trong suốt khi clk thấp, rồi AND) tạo ra clk_mac. Enable: !rst_n || (fsm != IDLE) || !fifo_empty. D0 nối enable cố định bằng 1 (GATING_EN = 0); D1 dùng enable (GATING_EN = 1); hai bản build không khác nhau ở điểm nào khác.
Đầu ra / Quyết định / Thực thi	READ_RESULT trả các entry accumulator về một thanh ghi scalar; firmware cộng bias, requantize bằng clamp((acc·M + 2^(S−1)) >> S, 0, 127) và lấy argmax của 10 trong 12 logit đã đệm. Bài test kết thúc bằng một lệnh ghi vào SIM_CTRL.
Giao diện giám sát / Đánh giá	Ba môi trường UVM (mac_env, core_env, soc_env) với các golden model C gọi qua DPI-C (npu_ref, một instruction-set simulator RV32 và một model số nguyên bằng Python), SystemVerilog assertion, functional coverage, một agent retire-trace thụ động, bộ harness tương đương D0/D1 và một windowed VCD dump của engine để đếm toggle.



Hình 1. Kiến trúc hệ thống Mini-NPU (npu_top). Scalar core (màu tối) và MAC engine tách rời, có datapath chạy trên clock gated clk_mac (màu xanh ngọc), dùng chung DTCM hai cổng 64 KB. Khối SIM MMIO chỉ tồn tại trong mô phỏng.

3.3 Tính mới và đóng góp kỹ thuật kỳ vọng

Đóng góp 1. Một NPU RISC-V quy mô nhỏ chạy được bằng RTL: extension custom-0 instruction, command FIFO, engine outer-product int8 tách rời và DTCM hai cổng, chạy MLP MNIST khớp bit-exact với golden model.

Đóng góp 2. Một cơ chế clock gating mức module cùng phương pháp tương đương dựa trên bằng chứng: SHA-256 của retire trace, mcycle và minstret cuối, và output dump được so sánh giữa D0 và D1 cho mọi test mức soc và mọi lần chạy MNIST.

Đóng góp 3. Một quan hệ đo được giữa mức giảm switching activity (M1 và M2) và idle ratio của engine, được đối chiếu chéo trên MNIST với đường cong tổng hợp.

Đóng góp 4. Một hạ tầng kiểm chứng UVM tự xây dựng (ba môi trường, golden model DPI-C, assertion và coverage) cùng gói tái lập tạo lại kết quả bằng một lệnh cho mỗi flow.

Tính mới nằm ở sự kết hợp và ở bằng chứng, không phải ở một mạch gating mới (Mục 2.4).

3.4 Giả thuyết

Giả thuyết	Phát biểu kiểm chứng được	Bằng chứng cần có
H1	Thiết kế có gating D1 cho cùng hash retire trace, mcycle và minstret cuối, và output dump như D0 với mọi test mức soc, các workload tổng hợp và các lần chạy MNIST.	Equivalence harness: so sánh hash và bộ đếm theo từng test, với cùng binary, seed và simulator
H2	Mức giảm của M1 (số toggle trên clk_mac, tính trên 1,000 cycle) tăng theo idle ratio của engine và tiến tới tỉ lệ idle, trong khi mức giảm của M2 (mọi net của engine) nhỏ hơn và phụ thuộc vào data activity.	Đường cong E2 theo idle ratio 10, 25, 50, 75 và 90 %, D1 so với D0
H3	Workload MNIST, đặt tại idle ratio của chính nó (npu_busy_cyc / mcycle), nằm trong độ phân tán của các điểm E2 tổng hợp.	Số toggle E4 cho 100 ảnh MNIST so với đường cong E2
H4	Các môi trường UVM tự xây đóng 100 % bin coverage đã khai báo trên D0 với 0 mismatch ở scoreboard và 0 assertion fail.	Báo cáo coverage đã merge; log regression với số UVM_ERROR và UVM_FATAL

3.5 Phạm vi

TRONG PHẠM VI:

- Các extension M, Zicsr, Zifencei và Zbb của core, và bốn custom instruction cùng command FIFO của chúng.
- MAC engine (FIFO, operand buffer, PE array, accumulator, CSR), DTCM hai cổng, ITCM và simulation MMIO.
- Cell clock gating và cấu hình D1; equivalence harness D0/D1.
- Ba môi trường UVM, một golden model C của engine, một instruction-set simulator C và một golden model số nguyên của MLP.
- Firmware bare-metal và pipeline MNIST; flow đếm toggle và các hình của nó.

NGOÀI PHẠM VI / KHÔNG PHẢI MỤC TIÊU (để dành cho luận văn tốt nghiệp):

- RVV/Zve32x và extension F; DMA, nhiều tile accumulator và PE dạng pipeline.
- Synthesis, place-and-route, bring-up trên FPGA, và mọi tuyên bố công suất tuyệt đối hay ở mức gate; mọi con số công suất đều là ước lượng switching activity từ RTL.
- Compiler toolchain (IREE hoặc TensorFlow Lite Micro [19]) và nạp chương trình qua frontdoor (bus).
- Kiểm chứng có hỗ trợ LLM. Mảng 8×8 chỉ được xây nếu còn thời gian sau khi đạt các tiêu chí cốt lõi.

4 PHƯƠNG PHÁP LUẬN / KẾ HOẠCH HIỆN THỰC VÀ ĐÁNH GIÁ

Giải thích ý tưởng đề xuất sẽ được xây dựng, kiểm thử, so sánh và phân tích như thế nào.

4.1 Phương pháp nghiên cứu tổng thể

Giai đoạn	Hoạt động chính	Đầu ra / Bằng chứng
A. Xác định vấn đề và baseline	Chốt phạm vi; đóng băng custom ISA và interface của engine; kiểm tra công cụ (UVM, DPI-C, coverage); viết golden model của engine kèm unit test.	Spec với các quyết định D1–D24; interface đã đóng băng; kết quả kiểm tra công cụ
B. Thiết kế	Mở rộng core (M, Zicsr, Zifencei, Zbb, custom-0, retire trace); viết DTCM hai cổng, MAC engine D0 và SoC top; viết nền firmware.	RTL; ghi chú theo khối; kết quả directed test và regression
C. Hiện thực và kiểm chứng	Xây mac_env, core_env và soc_env; viết sequence constrained-random (từ 200 seed trở lên), assertion và coverage; thêm clock gate (D1) và equivalence harness.	Log regression; báo cáo coverage đã merge; bảng tương đương D0/D1 (SC1–SC3)
D. Thực nghiệm	Chạy quét idle ratio tổng hợp và các lần chạy MNIST trên D0 và D1; chỉ dump VCD trong cửa sổ đo.	File CSV số toggle; logit MNIST so với golden model (SC4, SC5)
E. Phân tích	Tính phần trăm thay đổi của M1 và M2; đối chiếu chéo MNIST với đường cong tổng hợp; ghi lại hạn chế; viết báo cáo và gói tái lập.	Hình và bảng; báo cáo tiếng Anh; slide; tái tạo bằng một lệnh (SC6)

4.2 Kế hoạch hiện thực

Bước	Đầu vào	Xử lý / Hành động thiết kế	Đầu ra
1	Tài liệu CoralNPU; yêu cầu môn học	Nghiên cứu thiết kế mở; viết spec, decision log và tiêu chí thành công SC1–SC6	Spec đã duyệt; ISA và interface đã đóng băng
2	Spec mục 6, 7, 9	Viết các golden model: npu_ref (C), một instruction-set simulator RV32 (C) và một model MLP số nguyên (Python); chặn mỗi model bằng unit test riêng	Các oracle đã qua unit test của chúng
3	Core RV32I 5 tầng có sẵn	Thêm M, Zicsr, Zifencei, Zbb, giải mã và đẩy custom-0, ITCM và DTCM, và cổng retire-trace; đối chiếu với một ISS tham chiếu bằng Python	Regression của core xanh
4	Interface engine đã đóng băng	Viết command FIFO, FSM, các trạng thái LOAD và MAC, PE array, CSR và reset; đo độ trễ theo từng command	MAC engine D0; directed suite xanh
5	RTL core và engine	Tích hợp chúng trong SoC top; viết test firmware và các chương trình tổng hợp và MNIST	SoC smoke test, hiện 16/16 trên D0 và D1
6	Kế hoạch directed và random; coverage plan	Xây các môi trường UVM; random sequence trên từ 200 seed trở lên; assertion A-MAC-1..5; đóng coverage trên D0	Bằng chứng SC1 và SC2
7	Thiết kế D0	Thêm cell clock gate và logic enable; chạy mọi test soc và mọi lần chạy MNIST trên D0 và D1; so sánh hash retire-trace, mcycle, minstret và logit	Bảng tương đương SC3
8	D0 và D1 đã tương đương	Chạy quét tổng hợp và lần chạy MNIST đếm toggle; đếm toggle trong cửa sổ đo	Dữ liệu SC4, đồ thị và đối chiếu chéo
9	Mọi flow	Làm cho mỗi flow chạy được bằng một lệnh; ghi lại phiên bản công cụ, seed và git hash; thử trên một bản clone sạch	Gói tái lập SC6

4.3 Nền tảng phần cứng và phần mềm

Thành phần / Công cụ	Phiên bản / Đặc tả	Mục đích trong dự án
Nền tảng xử lý	PC Windows với WSL2 (Ubuntu 24.04)	Máy chủ cho QuestaSim và cho các công cụ Linux của regression
Cảm biến / Giao diện	Không áp dụng (chỉ mô phỏng)	Không bring-up board trong dự án này
Hệ điều hành / RTOS	Windows với WSL2 Ubuntu 24.04; CI trên Ubuntu 24.04. Không có RTOS: firmware bare-metal	Cùng một môi trường Linux ở máy cục bộ và trong continuous integration
Ngôn ngữ lập trình	SystemVerilog và Verilog-2001 (RTL, testbench); C99 (golden model, DPI-C); Python 3; assembly và C cho RISC-V (firmware, `-march=rv32im_zicsr_zifencei_zbb`)	RTL và kiểm chứng; golden model và firmware; huấn luyện, xuất dữ liệu và đếm toggle
Framework / Thư viện	UVM 1.2 (IEEE 1800.2 [16]); numpy; PyTorch (chỉ để huấn luyện); pytest	Phương pháp kiểm chứng; tham chiếu số nguyên và unit test của nó; huấn luyện MLP
Công cụ phát triển / Mô phỏng	QuestaSim 10.7c; Verilator 5.020 (lint); Icarus Verilog 12 (regression của core); RISC-V GNU toolchain; GitHub Actions	Mô phỏng, coverage và VCD dump; lint; regression của core; biên dịch firmware; regression tự động ở mỗi lần push

4.4 Dữ liệu, workload, benchmark hoặc kịch bản kiểm thử

Mục	Định nghĩa
Dataset / benchmark / workload	(1) MNIST [9] với MLP 784-64-32-10 được lượng tử hóa int8. (2) Firmware tổng hợp xen kẽ các đợt NPU (LOAD_WEIGHT, LOAD_INPUT và COMPUTE với K = 256) với các vòng lặp bận của scalar. (3) Các test directed và constrained-random cho engine, core và SoC.
Kích thước / số mẫu / thời lượng	MNIST: 100 ảnh (25 batch 4 ảnh) cho lần chạy activity, 1,000 ảnh test đầu cho phép so sánh bit-exact trên D0 và D1, và tập test 10,000 ảnh một lần trên D0. Tổng hợp: năm mức idle ratio (10, 25, 50, 75, 90 %). Test random: từ 200 seed trở lên cho mỗi môi trường.
Tiền xử lý / chuẩn hóa / biên dịch	Pixel sang int8 bằng (p·254 + 255) // 510. Weight int8 đối xứng trong ±127, bias int32. Requantization theo từng lớp với hệ số nhân nguyên M và độ dịch S chọn là S lớn nhất thỏa bound·M + 2^(S−1) ≤ $2^{31}-1$. Lớp cuối được đệm từ 10 lên 12 output; argmax bỏ qua phần đệm.
Chia train / validation / test	Huấn luyện trên tập train 60,000 ảnh (seed 0); 10,000 ảnh train dùng để hiệu chỉnh scale của activation (phân vị 99.99); đánh giá trên tập test 10,000 ảnh. Các phép so sánh phần cứng chỉ dùng ảnh test.
Điều kiện vận hành / dải đầu vào	Mô phỏng RTL một miền clock, không back-annotate timing. D0 và D1 dùng cùng binary, cùng phiên bản simulator và cùng tùy chọn dump; seed random được đặt tường minh.
Ground truth / đầu ra tham chiếu	Các golden model độc lập (engine, instruction-set simulator, MLP số nguyên) và retire trace của core; các model chỉ được dùng sau khi qua unit test của chính chúng.

4.5 Baseline và phương pháp so sánh

Baseline	Lý do chọn	Cấu hình / Kiểm soát công bằng
Baseline 1: D0 (GATING_EN = 0)	Cùng hierarchy và netlist với D1 ngoại trừ phần nối enable; là chuẩn tham chiếu cho tính tương đương và cho switching activity	Cùng binary, seed, phiên bản simulator và cửa sổ dump; mỗi cấu hình chạy một lần, chỉ chạy lại sau một lỗi bên ngoài đã được ghi lại
Baseline 2: các golden model độc lập	Oracle cho tính đúng chức năng; so sánh bit-exact loại bỏ tranh luận về dung sai	Chỉ dùng sau khi qua unit test của chúng; đối chiếu chéo với nhau (model MLP số nguyên với model engine, instruction-set simulator với một ISS tham chiếu Python)
Baseline 3: MLP float	Tham chiếu accuracy cho model int8 [1]	Cùng weight đã huấn luyện, cùng 10,000 ảnh test

4.6 Thước đo đánh giá

Khía cạnh	Thước đo ứng viên	Thước đo được chọn và cách đo
Chất lượng / ML	Accuracy, mức khớp nhãn	Accuracy của int8 và float trên tập test; logit và nhãn khớp bit-exact với golden model
Hiệu năng	Cycle, độ trễ, thông lượng	mcycle và minstret; số cycle bận cho mỗi command của engine (chặn trên K/4 + 3); idle ratio = npu_busy_cyc / mcycle; D0 và D1 phải khớp chính xác
Tính toán / Bộ nhớ	MAC mỗi cycle, dung lượng bộ nhớ	Đỉnh 64 MAC mỗi cycle; vùng dữ liệu DTCM 56,816 B trên 65,536 B
Phần cứng	LUT, FF, diện tích, tần số	Không đánh giá (không có synthesis)
Năng lượng	Công suất, năng lượng mỗi lần suy luận	Ước lượng switching activity: M1 = số toggle trên clk_mac; M2 = tổng số toggle trên mọi net của engine, tách thành phần clock tự do và phần clock gated; cả hai tính trên 1,000 cycle
Độ tin cậy	Mismatch, assertion fail, coverage	Số mismatch ở scoreboard và assertion fail (mục tiêu 0); functional coverage đã merge (mục tiêu 100 % bin đã khai báo)

4.7 Thiết kế thí nghiệm

Thí nghiệm	Mục tiêu	Biến độc lập	Đại lượng đo	Quyết định / So sánh
E1: Kiểm chứng chức năng	Chứng minh D0 và D1 đúng so với golden model	Test (directed, random seed); cấu hình D0 hoặc D1	Số mismatch ở scoreboard; số assertion fail; coverage %	Chỉ chấp nhận khi 0 mismatch, 0 failure và 100 % bin đã khai báo (SC1, SC2)
E2: Quét activity tổng hợp	Liên hệ switching activity với idle ratio	Idle ratio 10, 25, 50, 75, 90 %; cấu hình	M1 và M2 trên 1,000 cycle; % thay đổi D1 so với D0	Báo cáo đường cong; kiểm tra H2; chỉ chạy sau khi E3 pass
E3: Tương đương D0/D1	Chứng minh gating không đổi timing lẫn kết quả	Workload (mọi test soc, tổng hợp, MNIST); cấu hình	SHA-256 của retire log; mcycle; minstret; logit dump	Bất kỳ khác biệt nào cũng là fail (SC3)
E4: MNIST	Chứng minh suy luận bit-exact và đo activity của nó	Số ảnh (100, 1,000, 10,000); cấu hình	Logit và nhãn so với golden; accuracy; M1, M2; idle ratio	Bit-exact (SC5); đặt kết quả lên đường cong E2 (H3)

4.8 Kế hoạch phân tích và tái lập

- Mọi lần mô phỏng chạy qua một Python runner duy nhất (có tùy chọn cho simulator, test target, cấu hình D0 hoặc D1, seed và coverage); log được giữ theo từng lần chạy trong một thư mục theo ngày kèm `manifest.yaml` ghi lại phiên bản QuestaSim và loại licence, Verilator, GCC và Python, git hash, seed và SHA-256 của từng log.
- Seed random được đặt tường minh (ví dụ 1..200). Các golden model được chặn cổng: mỗi model phải qua unit test của chính nó trước khi dùng làm oracle, và các testbench được kiểm tra bằng cách chèn lỗi và xác nhận rằng các phép kiểm tra fail.
- Activity dump chỉ được lấy trong cửa sổ do firmware điều khiển (MEAS_START và MEAS_STOP) và chỉ cho scope của engine. Một script chuyển VCD sang CSV và một script thứ hai vẽ hình. Mỗi hình ghi rõ cấu hình, workload, seed và đơn vị, và được gắn nhãn “ước lượng switching activity”.
- Bằng chứng chức năng (UVM) và bằng chứng công suất (toggle) được báo cáo ở các phần riêng. Không tạo con số activity D1-so-với-D0 nào trước khi đạt tiêu chí tương đương (SC3).
- Một lệnh cho mỗi flow tái tạo dữ liệu regression, coverage và activity; điều này được kiểm trên một bản clone sạch ở tuần 15 (SC6).

4.9 Rủi ro kỹ thuật và giảm thiểu

Rủi ro	Tác động	Giảm thiểu / Phương án dự phòng
Licence của simulator thiếu một tính năng (UVM, randomize, covergroup, DPI-C)	Flow UVM bị chặn	Đã kiểm ở tuần 1: QuestaSim 10.7c chạy được cả bốn. Dự phòng: Vivado xsim (khi đó đã xác minh), Verilator hoặc pyuvm.
Mô phỏng quá chậm cho các batch MNIST	SC5 hoặc E4 không chạy xong	Đo ở tuần 14. Thứ tự cắt giảm: lần chạy 10,000 ảnh trước, sau đó E4 xuống 20 ảnh; SC3 trên MNIST và báo cáo không bao giờ bị cắt. Các lần chạy dài có thể chuyển sang Verilator kèm đối chiếu chéo trên QuestaSim.
Gating làm đổi timing (không giống hệt từng cycle)	SC3 fail	Theo cấu trúc, cạnh clock gated đầu tiên đến đúng lúc datapath cần; assertion A-GATE-1 cấm gating khi còn việc chờ; dự phòng là giống về kết quả, kèm lý do được ghi lại.
Interface giữa hai track thay đổi	Làm lại ở cả hai phía	Interface đóng băng ở tuần 2 (hai sinh viên đã ký ngày 29/10/2026); thay đổi sau đó cần cả hai chữ ký và cập nhật spec.
Golden model và RTL cùng hiểu sai spec	Kết quả sai bị chấp nhận	Các model độc lập bằng C và Python đối chiếu chéo với nhau; chèn lỗi vào testbench; các case unit tính tay.
Firmware ghi đè dữ liệu DTCM mà một LOAD đang xếp hàng còn đọc	Dữ liệu đầu vào hỏng	Quy tắc phần mềm (xả bằng READ_RESULT trước khi ghi đè) cộng một assertion và một negative test chứng minh assertion hoạt động.
Áp lực tiến độ	Sản phẩm bàn giao trễ	Thứ tự ưu tiên: D0 đúng, rồi D1 tương đương, rồi switching activity, rồi demo MNIST; mảng 8×8 và PE pipeline bị bỏ trước.

5 KẾT QUẢ KỲ VỌNG

Xác định các kết quả đo được, đóng góp kỳ vọng và sản phẩm bàn giao cụ thể của dự án.

5.1 Kết quả kỹ thuật kỳ vọng

Thiết kế D0 đã kiểm chứng được kỳ vọng pass toàn bộ regression với 0 mismatch ở scoreboard và 0 assertion fail, và đóng 100 % bin functional coverage đã khai báo. Thiết kế có gating D1 được kỳ vọng giống hệt D0 từng cycle trên mọi test mức soc và mọi lần chạy MNIST, đây là điều kiện để báo cáo bất kỳ con số activity nào. MLP int8 784-64-32-10 được kỳ vọng cho logit và nhãn khớp bit-exact với golden model trên 1,000 ảnh test đầu, và tái tạo đúng accuracy của golden trên lần chạy đủ 10,000 ảnh. Về switching activity, M1 được kỳ vọng giảm xấp xỉ tỉ lệ thuận với tỉ lệ idle, nên idle ratio cao loại bỏ phần lớn toggle của clk_mac, trong khi M2 được kỳ vọng giảm ít hơn và phụ thuộc vào data activity. Không đặt trước phần trăm mục tiêu. Điểm MNIST được kỳ vọng nằm trong độ phân tán của đường cong tổng hợp tại idle ratio của chính nó.

5.2 Bảng hiệu năng mục tiêu

Thước đo	Baseline / Giá trị hiện tại	Mục tiêu kỳ vọng	Điều kiện đo
Thước đo chính: tương đương D0/D1	Chỉ có D0: clk_mac toggle ở mọi cycle. Các chương trình smoke trên D0 và D1 giống hệt từng cycle (16/16, log dự án 10/11/2026).	Hash retire-trace, mcycle, minstret và output giống hệt cho 100 % test soc và lần chạy MNIST	Cùng binary, seed, simulator; E3
Thước đo phụ: switching activity	Số toggle D0 trên 1,000 cycle ở từng idle ratio (sẽ đo)	% thay đổi của M1 và M2 được báo cáo ở idle ratio 10–90 % và cho MNIST; không đặt trước mục tiêu	Cửa sổ VCD từ MEAS_START đến MEAS_STOP, scope engine; E2 và E4
Thước đo chức năng: mismatch và coverage	mac_env trên D0 và D1: 224/224 lần chạy đúng kỳ vọng, 81,332 phép so sánh ở scoreboard, 0 mismatch, 100 % của 7 covergroup (log dự án, 10/11/2026)	Kết quả tương tự cho core_env và soc_env; 0 mismatch và 100 % bin đã khai báo	QuestaSim 10.7c, UVM 1.2, runner với `--cov`; E1
Thước đo MNIST	Accuracy int8 trên tập test 97.43 % (float 97.48 %) trên 10,000 ảnh	Logit và nhãn của RTL khớp bit-exact trên 1,000 ảnh đầu (D0, D1); lần chạy đủ tái tạo accuracy của golden	Golden dump so với LOGITS dump của RTL; E4
Thước đo tái lập	Có manifest cho từng lần chạy; một lệnh cho regression của engine	Một lệnh cho mỗi flow, được kiểm trên một bản clone sạch	Tuần 15; SC6

5.3 Đóng góp khoa học / kỹ thuật kỳ vọng

- Một NPU RISC-V nhỏ, được giải thích đầy đủ (extension custom instruction, engine int8 tách rời, DTCM hai cổng), chạy MLP MNIST đã lượng tử hóa khớp bit-exact.
- Một phương pháp tương đương cho clock gating, được chứng minh ở mức module bằng hash retire-trace và so sánh bộ đếm, thay vì suy ra từ cấu trúc mạch.
- Một quan hệ đo được giữa idle ratio của engine NPU và switching activity được loại bỏ nhờ gating, kèm đối chiếu chéo trên workload thật.
- Một môi trường UVM tự xây dựng, tái sử dụng được, với golden model, assertion, coverage và gói tái lập, phục vụ các công việc sau như luận văn tốt nghiệp.

5.4 Sản phẩm bàn giao kỳ vọng

Sản phẩm	Nội dung tối thiểu / Bằng chứng chấp nhận
Đề cương nghiên cứu	Vấn đề, khoảng trống, ý tưởng, phương pháp luận và kết quả kỳ vọng đã được duyệt

Sản phẩm	Nội dung tối thiểu / Bằng chứng chấp nhận
Tổng quan tài liệu	Tổng hợp có phản biện các công trình liên quan (Phần 2) và phần công trình liên quan của báo cáo cuối
Sản phẩm thiết kế	Spec thiết kế kèm decision log, ghi chú theo khối cho core và MAC engine, hình kiến trúc và các định nghĩa interface
Prototype	RTL của core, MAC engine, DTCM hai cổng và SoC top ở D0 và D1; firmware; golden model
Gói thí nghiệm	Các môi trường UVM, danh sách test, runner, log kèm manifest, báo cáo coverage và file CSV số toggle
Báo cáo kỹ thuật	Báo cáo tiếng Anh với các chương kiểm chứng, switching activity và MNIST, thảo luận hạn chế và kết luận
Thuyết trình cuối và demo	Regression trực tiếp, tương đương D0/D1, suy luận MNIST và các đồ thị activity
Repository	Repository GitHub chứa RTL, testbench, model, firmware, script, tài liệu và gói tái lập

5.5 Diễn giải các kết quả âm tính hoặc hỗn hợp

- Nếu D1 không giống hệt D0 từng cycle, dự án lùi về phương án giống về kết quả và ghi lại chỗ cycle được thêm vào cùng lý do. Khi đó nghiên cứu cho thấy chi phí của gating thay vì việc nó không tốn chi phí.
- Nếu lợi ích của gating nhỏ ở idle ratio thấp, điều này phù hợp với mô hình rằng gating không giúp được một engine đang bận. Đường cong đo được khi đó cho thấy điểm hòa vốn, và idle ratio thấp của MNIST cho thấy workload này giữ engine bận và gating mức module là công cụ không phù hợp với nó.
- Nếu M2 giảm ít hơn nhiều so với M1, data activity bên trong engine đang chiếm ưu thế, điều này chỉ ra các kỹ thuật mức toán hạng như operand isolation là hướng công việc tương lai.
- Mọi con số activity đều đến từ mô phỏng RTL của một thiết kế và một simulator. Chúng là bằng chứng về thiết kế này, không phải tuyên bố về công suất trên silicon hay về các NPU nói chung.

Tiến độ dự án và các mốc

Ngày tháng theo lịch của dự án: tuần 1 bắt đầu thứ Hai 12/10/2026 và tuần 16 kết thúc Chủ nhật 31/01/2027. Trạng thái tính đến 11/11/2026 (tuần 5).

Tuần	Hoạt động	Mốc / Sản phẩm bàn giao (trạng thái 11/11/2026)
1 12–18/10	Tool gate: QuestaSim UVM, DPI-C, coverage, VCD scope, toolchain; xác định phạm vi vấn đề	Kết quả tool gate. *Đã xong 18/10/2026.*
2 19–25/10	Tài liệu ISA và CSR; interface của engine; golden model của engine kèm unit test	ISA và interface đã đóng băng. *Golden model xong 17/10/2026; interface được hai bên ký 29/10/2026.*
3–4 26/10–8/11	M, Zicsr, Zifencei, Zbb, giải mã và đẩy custom-0; instruction-set simulator; MAC engine D0; DTCM hai cổng	Kết thúc Phase 1. *Regression core 1,291/1,291; engine D0 và D1 xanh trên QuestaSim (5/11/2026); còn một quyết định chính sách của core (W2-1) chưa chốt.*
5–6 9–22/11	Tích hợp SoC và nền firmware; directed test cho mac_env và core_env	Soc test đầu tiên. *Tích hợp phía engine xong (10/11/2026); mac_env directed và random xong (7/11/2026).*
7–9 23/11–13/12	Test random và coverage; test soc; đóng coverage	SC1 và SC2 trên D0; bản nháp chương kiểm chứng. *mac_env: 100 % của 7 covergroup (10/11/2026); core_env và soc_env còn chờ.*
10–11 14–27/12	Cell clock gate, regression D1, equivalence harness	SC3 trên mọi test soc

Tuần	Hoạt động	Mốc / Sản phẩm bàn giao (trạng thái 11/11/2026)
12 28/12–3/1	Firmware tổng hợp; script đếm toggle; chạy E2	Dữ liệu và đồ thị E2 (SC4, phần tổng hợp)
13–14 4–17/1	Firmware MNIST; đóng gói weight và golden file; MNIST 1,000 ảnh trên D0 và D1; khung báo cáo	SC5 (bit-exact trên D0 và D1)
15 18–24/1	Chạy E4 đếm toggle MNIST (100 ảnh); chạy đủ 10,000 ảnh trên D0; tái tạo trên bản clone sạch	SC4 (MNIST), SC6
16 25–31/1	Báo cáo tiếng Anh, slide, gói tái lập; kiểm tra cuối và demo	Báo cáo cuối, demo và repository

Tài liệu tham khảo

[1] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, Salt Lake City, UT, USA, 2018, pp. 2704–2713.

[2] N. P. Jouppi *et al.*, "In-datacenter performance analysis of a Tensor Processing Unit," in *Proc. 44th Annu. Int. Symp. Computer Architecture (ISCA)*, Toronto, ON, Canada, 2017, pp. 1–12.

[3] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, Jan. 2017.

[4] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proc. IEEE*, vol. 105, no. 12, pp. 2295–2329, Dec. 2017.

[5] M. Horowitz, "Computing's energy problem (and what we can do about it)," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, 2014, pp. 10–14.

[6] Google, "Coral NPU," GitHub repository *google-coral/coralnpu*, commit 382b5c1, and Coral NPU developer guides (developers.google.com/coral/guides), read Oct. 15, 2026.

[7] H. Genc *et al.*, "Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration," in *Proc. 58th ACM/IEEE Design Automation Conf. (DAC)*, San Francisco, CA, USA, 2021, pp. 769–774.

[8] A. Waterman and K. Asanović, Eds., *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, Document Version 20191213, RISC-V Foundation, Dec. 2019.

[9] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.

[10] J. L. Hennessy and D. A. Patterson, "A new golden age for computer architecture," *Commun. ACM*, vol. 62, no. 2, pp. 48–60, Feb. 2019.

[11] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware/Software Interface*, 2nd ed. Morgan Kaufmann, 2021.

[12] Q. Wu, M. Pedram, and X. Wu, "Clock-gating and its application to low power design of sequential circuits," *IEEE Trans. Circuits Syst. I, Fundam. Theory Appl.*, vol. 47, no. 3, pp. 415–420, Mar. 2000.

[13] L. Benini and G. De Micheli, "Automatic synthesis of low-power gated-clock finite-state machines," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 15, no. 6, pp. 630–643, Jun. 1996.

[14] M. Keating, D. Flynn, R. Aitken, A. Gibbons, and K. Shi, *Low Power Methodology Manual: For System-on-Chip Design*. New York, NY, USA: Springer, 2007.

[15] F. N. Najm, "A survey of power estimation techniques in VLSI circuits," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 2, no. 4, pp. 446–455, Dec. 1994.

[16] *IEEE Standard for Universal Verification Methodology Language Reference Manual*, IEEE Std 1800.2-2017, 2017.

[17] *IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language*, IEEE Std 1800-2017, 2018.

[18] W. Stallings, *Computer Organization and Architecture: Designing for Performance*, 11th ed. Pearson, 2019.

[19] R. David *et al.*, "TensorFlow Lite Micro: Embedded machine learning for TinyML systems," in *Proc. Mach. Learn. Syst. (MLSys)*, vol. 3, 2021, pp. 800–811.